Pruning neural networks without any data by iteratively conserving synaptic flow

Hidenori Tanaka et al (Standford), NeurIPS 2020

Hojoon Lee

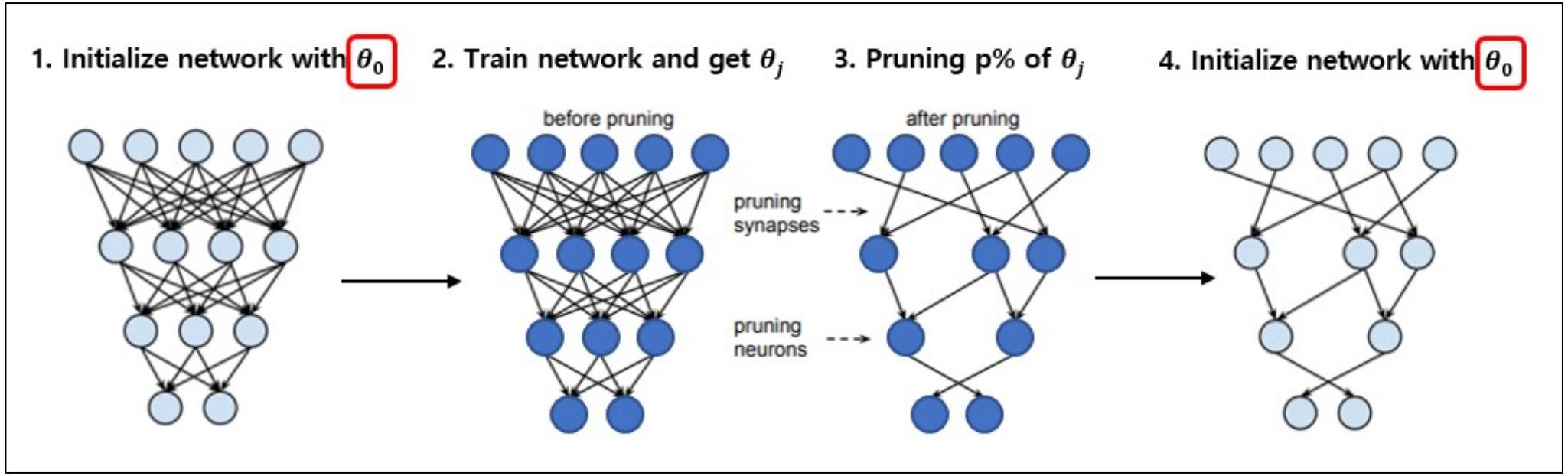
2023.4.12

Table of Contents

- Related Work & Preliminaries.
- Problem of previous work: Layer Collapse.
- Synaptic Saliency Conservation Theorem. Theorem that guarantees to prevent Layer Collapse.
- SynFlow: Algorithm that satisfies Synaptic Saliency Conservation Theorem.
- Experiments.
- Conclusion.
- (Thoughts) Future Research Direction

Related Work

- **Pruning after training**
  - Assign <u>scores</u> to parameters after training and remove the parameters with the lowest scores.
    - Scores: weight magnitude [1], gradient magnitude [2].
  - (+) compress neural networks at test time.
  - (–) no reduction in the cost of training.



[1] The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks., Jonathan Frankle et al., ICLR 2019.
[2] SNIP: Single-Shot Network Pruning based on Connection Sensitivity. Namhoon Lee et al., ICLR 2019.

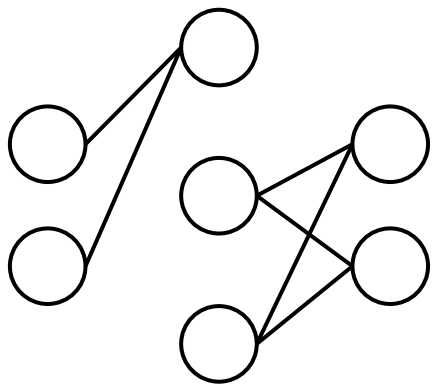
Related Work

- **Pruning before training**
  - Assign scores to parameters before training and remove the parameters with the lowest scores.
  - (+) compress neural networks at both train & test time.
  - (–) unstable & low performance compared to the pruning after training.
    - Why unstable?
      - **Layer Collapse**: Prunes all the pathways of the neural network, which makes network to be untrainable.

- **Contribution**
  - Identify Layer-Collapse of the existing pruning algorithms.
  - Propose a pruning before training algorithm, SynFlow, which theoretically prevents Layer-Collapse.
  - SynFlow does not require any data for pruning.
    - (I believe this will be highly beneficial in reinforcement learning setup)

Preliminaries

- **Definition**
  - **Layer Collapse**
    - Pruning algorithm prunes all the pathways of the neural network, which makes an entire network to be untrainable.
  - **Compression Ratio**
    - no.of original params / no.of pruned params.
  - **Maximum Compression Ratio** (ρ_{max})
    - Maximal possible compression ratio for a network that does not lead to collapse.
  - **Critical Compression** (ρ_{cr})
    - Maximal compression ratio for a given algorithm can achieve without inducing layer collapse.



Layer-Collapsed Network

Identifying Layer-Collapse

- Existence of Layer-Collapse

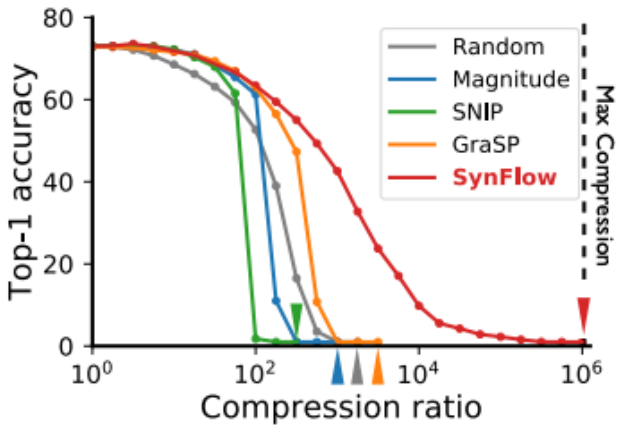


Figure 1: **Layer-collapse leads to a sudden drop in accuracy.** Top-1 test accuracy as a function of the compression ratio for a VGG-16 model pruned at initialization and trained on CIFAR-100. Colored arrows represent the critical compression of the corresponding pruning algorithm. Only our algorithm, SynFlow, reaches the theoretical limit of max compression (black dashed line) without collapsing the network. See

- Where does Layer-Collapse Occurs

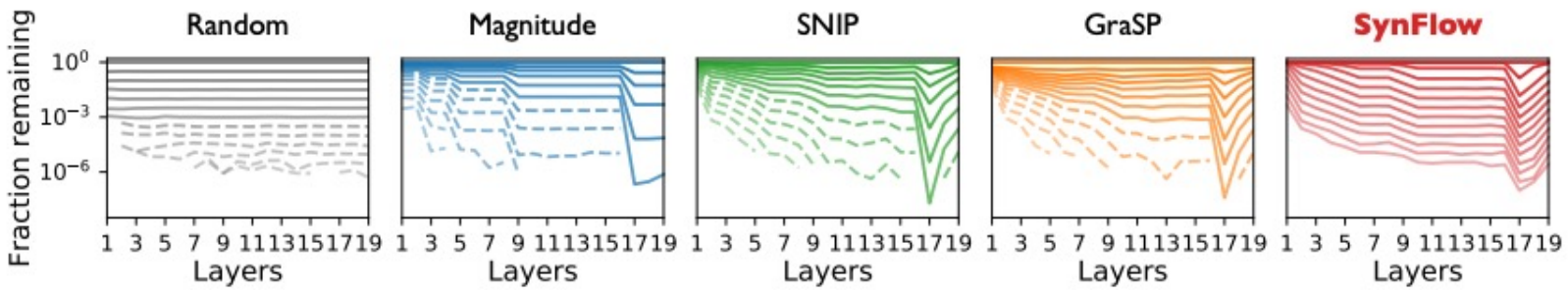


Figure 2: **Where does layer-collapse occur?** Fraction of parameters remaining at each layer of a VGG-19 model pruned at initialization with ImageNet over a range of compression ratios (10^n for $n = 0, 0.5, \ldots, 6.0$). A higher transparency represents a higher compression ratio. A dashed line indicates that there is at least one layer with no parameters, implying layer-collapse has occurred.

staircase pattern in Fig. 2. Magnitude pruning effectively prunes parameters based on the variance of their initialization, which for common network initializations, such as Xavier [37] or Kaiming [38], are inversely proportional to the width of a layer [34]. With magnitude pruning the *widest layers*, the layers with largest input or output dimensions, are the first to be fully pruned. Gradient-based pruning algorithms SNIP [13] and GraSP [14] also prune layers at different rates, but it is less clear what the root cause for this preference is. In particular, both SNIP and GraSP aggressively prune the *largest layer*, the layer with the most trainable parameters, evident by the sharp peaks in Fig. 2.

Wide layers are prone to collapse. (Due to the standard initialization scheme).

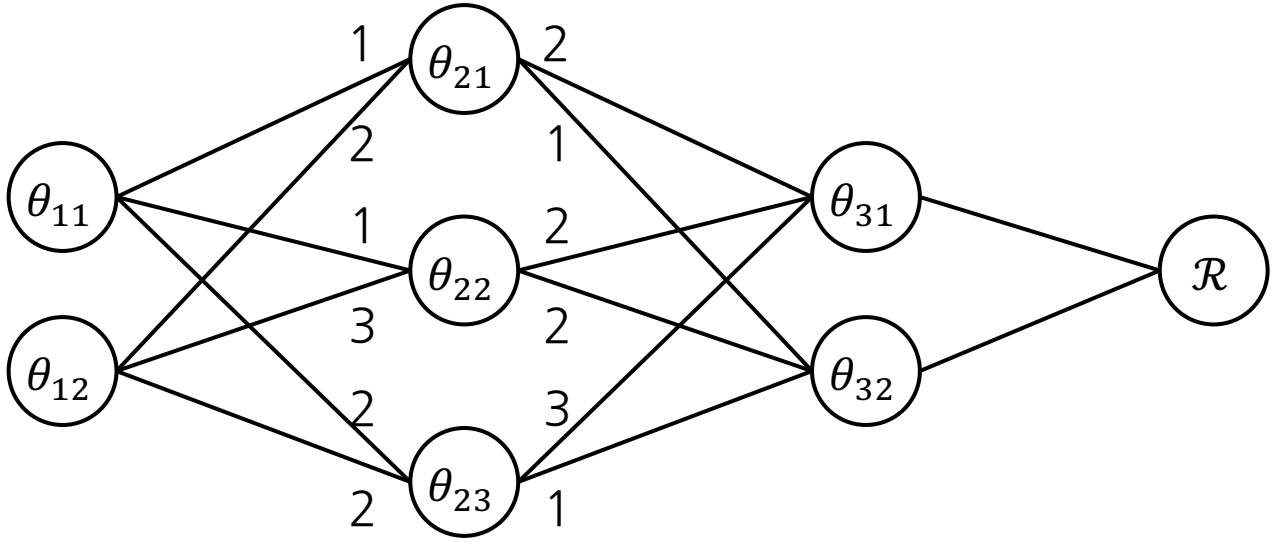
Conservation Laws of Synaptic Saliency

- Synaptic Saliency (시냅스, 뉴런 중요도)

A general class of gradient-based scores. *Synaptic saliency* is a class of score metrics that can be expressed as the Hadamard product

$$\mathcal{S}(\theta) = \frac{\partial \mathcal{R}}{\partial \theta} \odot \theta, \tag{1}$$

where $\mathcal{R}$ is a scalar loss function of the output y of a feed-forward network parameterized by θ. When



$S(\theta_{21}^{in}) = 1 + 2 = 3$ $\quad$ $S(\theta_{21}^{out}) = 2 + 1 = 3$

$S(\theta_{22}^{in}) = 1 + 3 = 4$ $\quad$ $S(\theta_{22}^{out}) = 2 + 2 = 4$

$S(\theta_{23}^{in}) = 2 + 2 = 4$ $\quad$ $S(\theta_{23}^{out}) = 3 + 1 = 4$

$S(\theta_{2}^{in}) = 3 + 4 + 4 = 11$

$S(\theta_{2}^{out}) = 3 + 4 + 4 = 11$

Conservation Laws of Synaptic Saliency

- Conservation Laws of Synaptic Saliency

Theorem 1. ***Neuron-wise Conservation of Synaptic Saliency.*** *For a feedforward neural network with continuous, homogeneous activation functions, $\phi(x) = \phi'(x)x$, (e.g. ReLU, Leaky ReLU, linear), the sum of the synaptic saliency for the incoming parameters (including the bias) to a hidden neuron ($\mathcal{S}^{in} = \langle \frac{\partial \mathcal{R}}{\partial \theta^{in}}, \theta^{in} \rangle$) is equal to the sum of the synaptic saliency for the outgoing parameters from the hidden neuron ($\mathcal{S}^{out} = \langle \frac{\partial \mathcal{R}}{\partial \theta^{out}}, \theta^{out} \rangle$).*

$$S(\theta_{21}^{in}) = S(\theta_{21}^{out})$$

Theorem 2. ***Network-wise Conservation of Synaptic Saliency.*** *The sum of the synaptic saliency across any set of parameters that exactly[3] separates the input neurons x from the output neurons y of a feedforward neural network with homogeneous activation functions equals $\langle \frac{\partial \mathcal{R}}{\partial x}, x \rangle = \langle \frac{\partial \mathcal{R}}{\partial y}, y \rangle$.*

$$S(\theta_{2}^{in}) = S(\theta_{2}^{out})$$

Theorem 3. ***Iterative, positive, conservative scoring achieves Maximal Critical Compression.*** *If a pruning algorithm, with global-masking, assigns positive scores that respect layer-wise conservation and if the prune size, the total score for the parameters pruned at any iteration, is strictly less than the cut size, the total score for an entire layer, whenever possible, then the algorithm satisfies the Maximal Critical Compression axiom.*

$$If S(\theta_i) > \sum_{p}^{p \in Pruned} S(\theta_p)$$

$If\ total\ score\ for\ an\ entire\ layer >\ the\ total\ saliency\ score\ for\ the\ pruned\ parameters$:

$Algorithm\ satisfies\ the\ Maximal\ Critical\ Compression\ axiom\ (= prevents\ layer\ collapse)$.

SynFlow: Algorithm that satisfies Maximal Critical Compression

- The Iterative Synaptic Flow Pruning (SynFlow) algorithm

$$\mathcal{R}_{\mathrm{SF}} = \mathbb{1}^T \left(\prod_{l=1}^{L} |\theta^{[l]}| \right) \mathbb{1}$$

A general class of gradient-based scores. *Synaptic saliency* is a class of score metrics that can be expressed as the Hadamard product

$$\mathcal{S}(\theta) = \frac{\partial \mathcal{R}}{\partial \theta} \odot \theta, \tag{1}$$

where $\mathcal{R}$ is a scalar loss function of the output y of a feed-forward network parameterized by θ. When

Iteratively prune the parameters based on SynFlow Saliency scores.

Algorithm 1: Iterative Synaptic Flow Pruning (SynFlow).

Input: network $f(x;\theta_0)$, compression ratio ρ, iteration steps n

0: $f(x;\theta_0)$; ▷Set model to eval mode[a]

1: $\mu = \mathbb{1}$; ▷Initialize binary mask

for k *in* $[1,\ldots,n]$ **do**

2: $\theta_\mu \leftarrow \mu \odot \theta_0$; ▷Mask parameters

3: $\mathcal{R} \leftarrow \mathbb{1}^T \left(\prod_{l=1}^{L} |\theta_\mu^{[l]}| \right) \mathbb{1}$; ▷Evaluate SynFlow objective

4: $\mathcal{S} \leftarrow \frac{\partial \mathcal{R}}{\partial \theta_\mu} \odot \theta_\mu$; ▷Compute SynFlow score

5: $\tau \leftarrow (1-\rho^{-k/n})$ percentile of $\mathcal{S}$; ▷Find threshold

6: $\mu \leftarrow (\tau < \mathcal{S})$; ▷Update mask

end

7: $f(x;\mu \odot \theta_0)$; ▷Return masked network

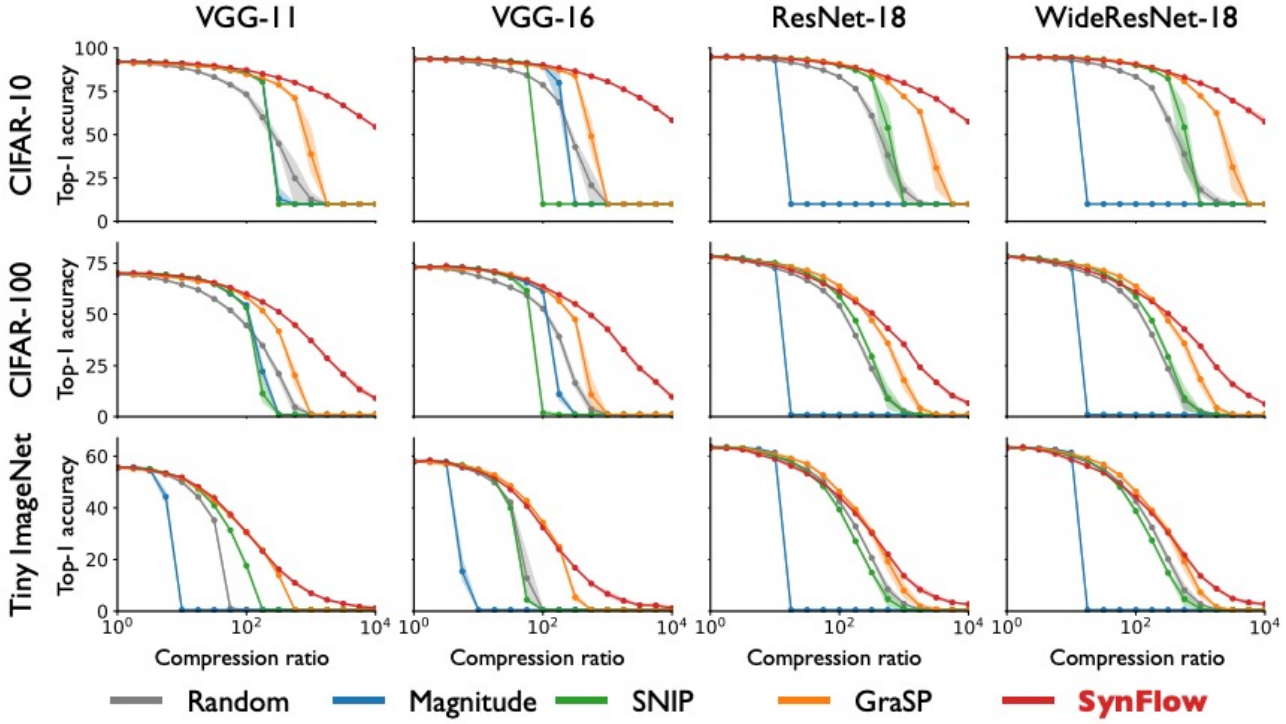
```
@torch.no_grad()
def linearize(model):
    # model.double()
    signs = {}
    for name, param in model.state_dict().items():
        signs[name] = torch.sign(param)
        param.abs_()
    return signs

signs = linearize(model)

(data, _) = next(iter(dataloader))
input_dim = list(data[0,:].shape)
input = torch.ones([1] + input_dim).to(device)
output = model(input)
torch.sum(output).backward()
```

Experiments

- Experimental Setup

Computational cost of SynFlow. The computational cost of a pruning algorithm can be measured by the number of forward/backward passes (#iterations × #examples per iteration). We always run the data-agnostic SynFlow with 100 iterations, implying it takes 100 passes no matter the dataset. SNIP and GraSP each involve one iteration, but use ten times the number of classes per iteration requiring 1000, 2000, and 10,000 passes for CIFAR-100, Tiny-ImageNet, and ImageNet respectively.

- Results



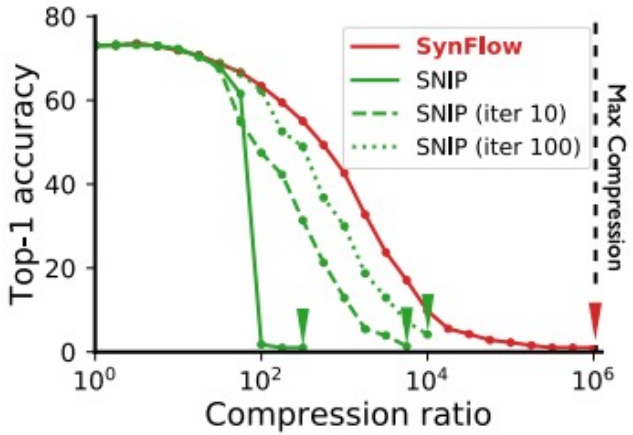


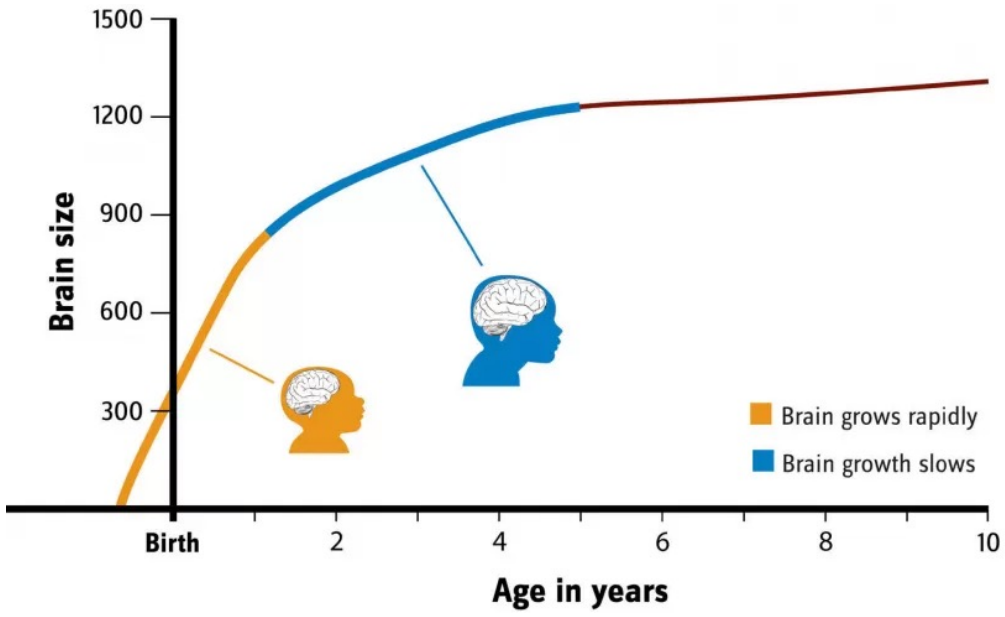
Figure 7: **Iteration improves SNIP, but layer-collapse still occurs.** Top-1 test accuracy as a function of the compression ratio for a VGG-16 model pruned at initialization and trained on CIFAR-100.

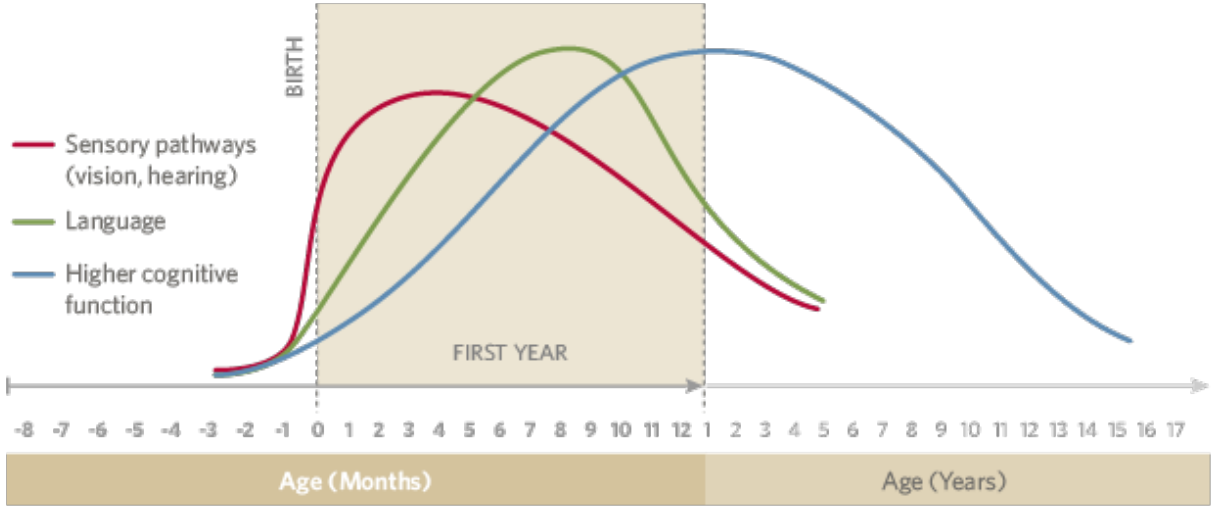
Conclusion

- Identified the problem of existing pruning algorithms, **Layer Collapse.**
- Iterative conservation of synaptic saliency can guarantee Maximal Compression ratio.
  - Maximal Compression ratio: Maximum compression ratio that prevents layer collapse.
- **SynFlow** can **prune the network without Layer Collapse**, even **without any dataset.**

(Thoughts) Future Research Direction

- **Progressive Neural Network**
  - Network progressively grows over time.
  - Possible Efficacy of Progressive Growing Neural Network in Reinforcement Learning.
    - Start from a small network → prevents overfitting.
    - Progressive growing → prevents plasticity loss.
      - Plasticity (=ability to fit a new target).





Source: Nelson, C. (2000). From *Neurons to Neighborhoods: The Science of Early Childhood Development*. Washington, DC: The National Academies Press.

(Thoughts) Future Research Direction

- **How can we progressively grow neural network?**
  - Isotropic architecture (e.g., Transformer)
    - Stack neural network [1].

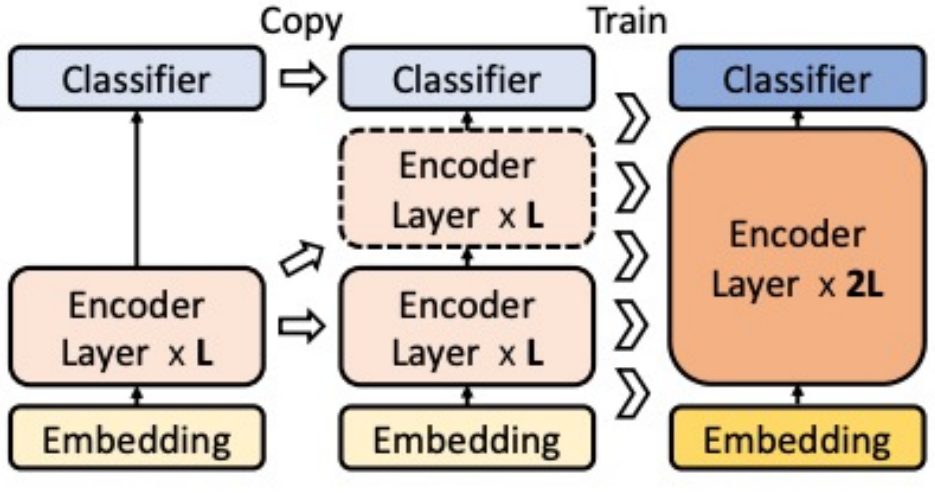
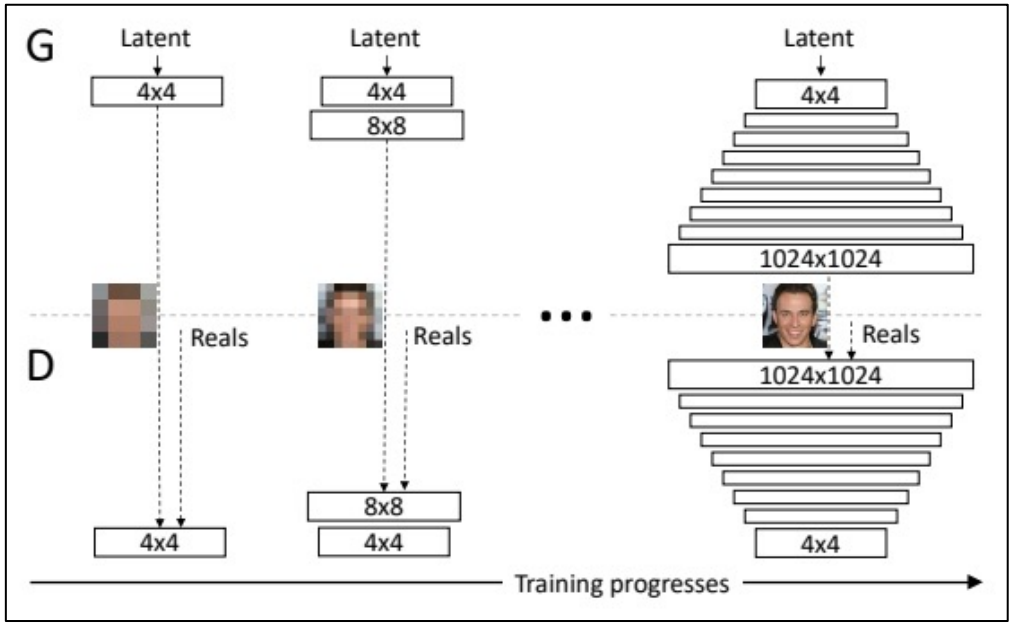


Figure 3. The diagram of the *stacking* algorithm.

  - Anisotropic architecture (e.g., Convolution):
    - Stack + Lower-Resolution [2].

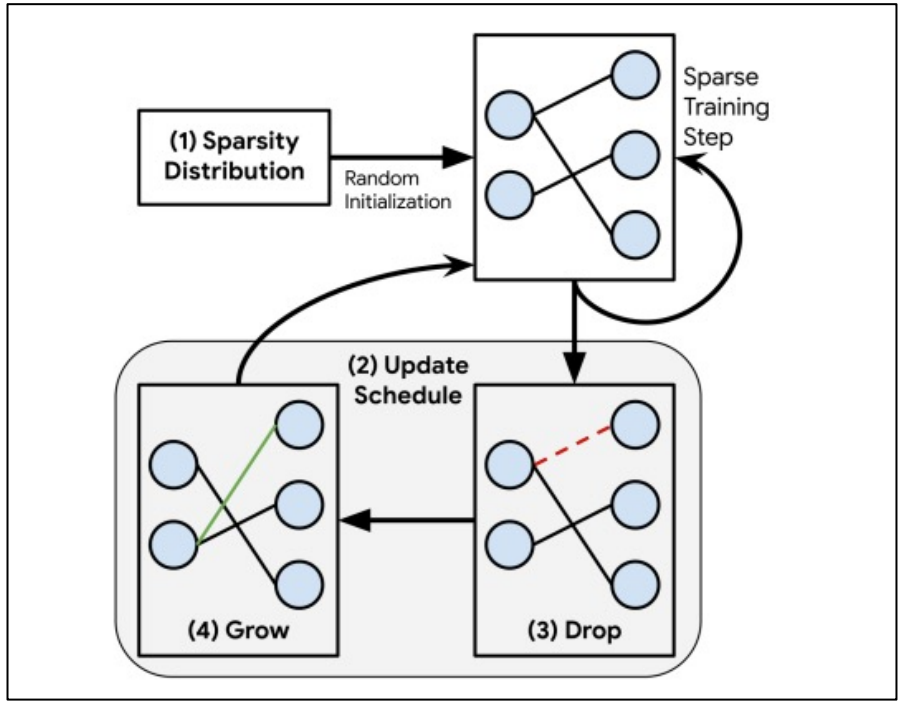


[1] Efficient Training of BERT by Progressively Stacking., Linyuan Gonget al., ICML 2019.
[2] Progressive Growing GANS. Tero Karraset al., ICLR 2018.

(Thoughts) Future Research Direction

- **Can we use Stack + Low-Resolution in RL?**
  - No! Low Resolution will not work in RL. RL requires fine-grained control.

- **How can Progressively Grow Neural Network in RL?**
  - What about sparse to dense neural network? (RigL [1])



Algorithm 1 RigL

Input: Network f_Θ, dataset D
Sparsity Distribution: $\mathbb{S} = \{s^1, \ldots, s^L\}$
Update Schedule: ΔT, T_{end}, α, f_{decay}
$\theta \leftarrow$ Randomly sparsify Θ using $\mathbb{S}$
for each training step t **do**
  Sample a batch $B_t \sim D$
  $L_t = \sum_{i \sim B_t} L((f_\theta(x_i),\ y_i)$
  if $t \pmod{\Delta T} == 0$ and $t < T_{end}$ **then**
    for each layer l **do**
      $k = f_{decay}(t;\ \alpha, T_{end})(1 - s^l)N^l$
      $\mathbb{I}_{active} = ArgTopK(-|\theta^l|,\ k)$
      $\mathbb{I}_{grow} = ArgTopK_{i \notin \theta^l \setminus \mathbb{I}_{active}}(|\nabla_{\Theta^l} L_t|,\ k)$
      $\theta \leftarrow$ Update connections θ using $\mathbb{I}_{active}$ and $\mathbb{I}_{grow}$
    end for
  else
    $\theta = \theta - \alpha \nabla_\theta L_t$

- SynFlow!
- Why? We usually do not have any dataset prior to training in RL.

- Why don't we progressively grow the neural network?

Q & A